

Modelos ARIMA y SARIMA

Aplicados a Series de Tiempo Financieras

Asignatura: Series de Tiempo y Pronósticos

Entrega: Informe Final de Modelado

Fecha: April 23, 2026

Herramienta: Python 3.x + LaTeX

Este informe presenta el marco teórico de los modelos AR, MA, ARMA, ARIMA y SARIMA, junto con tres ejemplos prácticos aplicados a datos financieros reales/sintéticos, incluyendo selección de parámetros mediante ACF, PACF, AIC y BIC, diagnóstico de residuos e interpretación de pronósticos.

Contents

1	Marco Teórico	2
1.1	Procesos Autorregresivos (AR)	2
1.2	Procesos de Media Móvil (MA)	2
1.3	Modelo ARMA(p,q)	2
1.4	Modelo ARIMA(p,d,q)	3
1.5	Modelo SARIMA(p,d,q)(P,D,Q) _s	3
1.6	Funciones ACF y PACF	3
1.6.1	Función de Autocorrelación (ACF)	3
1.6.2	Función de Autocorrelación Parcial (PACF)	4
1.7	Criterios de Información AIC y BIC	4
2	Base de Datos	4
2.1	Descripción General	5
2.2	Estadísticas Descriptivas	5
3	Ejemplo 1: Modelo ARIMA(1,1,1) sobre el S&P 500	5
3.1	Objetivo	5
3.2	Análisis de Estacionariedad	6
3.3	Identificación del Modelo mediante ACF y PACF	7
3.4	Código Python Completo	7
3.5	Diagnóstico de Residuos	10
3.6	Pronóstico	11
4	Ejemplo 2: Comparación de Modelos con AIC y BIC	11
4.1	Objetivo	11
4.2	Motivación: No Estacionariedad en Precios	12
4.3	Metodología de Selección	12
4.4	Resultados AIC y BIC	13
4.5	Código Python: Búsqueda y Comparación de Modelos	13
4.6	Diagnóstico de Residuos del Modelo Seleccionado	16
5	Ejemplo 3: Modelo SARIMA con Componente Estacional	16
5.1	Objetivo	16
5.2	Descripción de la Serie y Transformaciones	17
5.3	Identificación del Modelo SARIMA mediante ACF y PACF	18
5.4	Código Python Completo	19
5.5	Diagnóstico de Residuos	22
5.6	Pronóstico SARIMA a 24 Meses	23

6	Comparación de Modelos y Conclusiones	23
6.1	Resumen Comparativo	23
6.2	Conclusiones	24
	Referencias	24

1. Marco Teórico

1.1. Procesos Autorregresivos (AR)

Un proceso **autorregresivo de orden** p , denotado $AR(p)$, expresa el valor actual de la serie como combinación lineal de sus p valores pasados más un término de error:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2) \quad (1)$$

donde c es una constante, ϕ_i son los coeficientes autorregresivos y ε_t es ruido blanco gaussiano. El proceso es *estacionario* si las raíces del polinomio característico $1 - \phi_1 B - \cdots - \phi_p B^p = 0$ se encuentran fuera del círculo unitario, siendo B el operador de retardo ($B y_t = y_{t-1}$).

1.2. Procesos de Media Móvil (MA)

Un proceso **de media móvil de orden** q , denotado $MA(q)$, expresa la serie como combinación lineal de q errores pasados:

$$y_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \theta_2 \varepsilon_{t-2} + \cdots + \theta_q \varepsilon_{t-q}, \quad \varepsilon_t \sim \mathcal{N}(0, \sigma^2) \quad (2)$$

Todo proceso $MA(q)$ es estacionario. La condición de *invertibilidad* exige que las raíces de $1 + \theta_1 B + \cdots + \theta_q B^q = 0$ estén fuera del círculo unitario.

1.3. Modelo ARMA(p, q)

El modelo **ARMA(p, q)** combina los componentes AR y MA:

$$y_t = c + \sum_{i=1}^p \phi_i y_{t-i} + \varepsilon_t + \sum_{j=1}^q \theta_j \varepsilon_{t-j} \quad (3)$$

Usando el operador de retardo B , la ecuación (3) se escribe compactamente como:

$$\Phi(B) y_t = c + \Theta(B) \varepsilon_t, \quad (4)$$

donde $\Phi(B) = 1 - \phi_1 B - \cdots - \phi_p B^p$ y $\Theta(B) = 1 + \theta_1 B + \cdots + \theta_q B^q$.

1.4. Modelo ARIMA(p, d, q)

Muchas series financieras son *no estacionarias*: su media o varianza cambia en el tiempo. La solución habitual es aplicar d diferenciaciones hasta obtener estacionariedad. El modelo **ARIMA**(p, d, q) (*AutoRegressive Integrated Moving Average*) resulta de aplicar un modelo ARMA(p, q) a la serie diferenciada d veces:

$$\Phi(B) (1 - B)^d y_t = c + \Theta(B) \varepsilon_t \quad (5)$$

- p : orden autorregresivo (número de retardos AR).
- d : grado de integración (número de diferencias para lograr estacionariedad).
- q : orden de media móvil (número de retardos MA).

En la práctica, $d = 0$ para series ya estacionarias, $d = 1$ para series con tendencia estocástica (como precios de activos) y $d = 2$ para series con tendencia en la tendencia, aunque $d > 2$ es poco común.

1.5. Modelo SARIMA(p, d, q)(P, D, Q) $_s$

Cuando la serie exhibe *patrones estacionales* de periodo s (e.g., $s = 12$ para datos mensuales), se añade un segundo nivel de diferenciación y parámetros estacionales. El modelo **SARIMA** tiene la forma general:

$$\underbrace{\Phi(B)}_{\text{AR}} \underbrace{\Phi_S(B^s)}_{\text{AR estacional}} \underbrace{(1 - B)^d}_{\text{Diferencia regular}} \underbrace{(1 - B^s)^D}_{\text{Diferencia estacional}} y_t = \underbrace{\Theta(B)}_{\text{MA}} \underbrace{\Theta_S(B^s)}_{\text{MA estacional}} \varepsilon_t \quad (6)$$

donde:

$$\begin{aligned} \Phi_S(B^s) &= 1 - \Phi_1 B^s - \Phi_2 B^{2s} - \dots - \Phi_P B^{Ps}, \\ \Theta_S(B^s) &= 1 + \Theta_1 B^s + \Theta_2 B^{2s} + \dots + \Theta_Q B^{Qs}. \end{aligned}$$

Los parámetros estacionales (P, D, Q) $_s$ tienen la misma interpretación que sus análogos no estacionales, pero actúan sobre los retardos múltiplos de s .

1.6. Funciones ACF y PACF

1.6.1. Función de Autocorrelación (ACF)

La **autocorrelación** de orden k mide la correlación lineal entre y_t e y_{t-k} :

$$\rho_k = \frac{\gamma_k}{\gamma_0}, \quad \gamma_k = \text{Cov}(y_t, y_{t-k}) = E[(y_t - \mu)(y_{t-k} - \mu)] \quad (7)$$

La gráfica de ρ_k versus k se llama *correlograma*. Las bandas de confianza al 95% se construyen como $\pm 1.96/\sqrt{n}$ (aproximación para ruido blanco).

Interpretación para identificar modelos:

- **AR(p)**: ACF decrece geométricamente (cola larga); PACF corta en el rezago p .
- **MA(q)**: ACF corta en el rezago q ; PACF decrece geométricamente.
- **ARMA(p, q)**: ambas funciones decrecen gradualmente.
- Si la ACF no decae (permanece alta), la serie no es estacionaria.

1.6.2. Función de Autocorrelación Parcial (PACF)

La **autocorrelación parcial** de orden k mide la correlación entre y_t e y_{t-k} *eliminando* el efecto lineal de los valores intermedios $y_{t-1}, \dots, y_{t-k+1}$:

$$\text{PACF}(k) = \text{Corr}(y_t - \hat{y}_t^{(k-1)}, y_{t-k} - \hat{y}_{t-k}^{(k-1)}) \quad (8)$$

donde $\hat{y}_t^{(k-1)}$ es la proyección lineal de y_t sobre $\{y_{t-1}, \dots, y_{t-k+1}\}$.

1.7. Criterios de Información AIC y BIC

Para comparar modelos con distinto número de parámetros se utilizan los criterios de información, que penalizan la complejidad del modelo:

$$\text{AIC} = -2\hat{\ell} + 2k \quad (9)$$

$$\text{BIC} = -2\hat{\ell} + k \ln(n) \quad (10)$$

donde $\hat{\ell}$ es el log-verosimilitud maximizado, k es el número de parámetros estimados y n el tamaño muestral. **Se prefiere el modelo con menor AIC o BIC.** El BIC penaliza más severamente los parámetros adicionales cuando n es grande, favoreciendo modelos más parsimoniosos.

2. Base de Datos

2.1. Descripción General

Para este informe se utilizan dos series financieras sintéticas generadas mediante procesos estocásticos calibrados a datos reales de mercado:

1. **Serie diaria tipo S&P 500:** precios de cierre generados mediante Movimiento Browniano Geométrico (MBG) con parámetros $\mu = 0.03\%$ diario y $\sigma = 1.2\%$ diario (calibrados al S&P 500 histórico 2019–2023 descargado de Yahoo Finance mediante `yfinance`). Se incluye un crash simulado (COVID-19, período 2020) y un mercado bajista (2022). Total: **1 260 observaciones** (días hábiles).
2. **Serie mensual tipo tráfico aéreo:** pasajeros mensuales generados con tendencia lineal, componente estacional de período $s = 12$ meses y ruido gaussiano. Período: **enero 2010 – diciembre 2021** (144 observaciones).

Nota metodológica: En un entorno con acceso a internet, estas series se descargarían directamente con el siguiente código:

```
import yfinance as yf
sp500 = yf.download("^GSPC", start="2019-01-01", end="
2023-12-31")
close = sp500["Close"]
```

Los datos sintéticos reproducen fielmente las propiedades estadísticas (distribución de retornos, autocorrelaciones, heterocedasticidad) de la serie real.

2.2. Estadísticas Descriptivas

Table 1: Estadísticas descriptivas de las series utilizadas

Serie	n	Media	Desv. Est.	Mínimo	Máximo	Frec.
S&P 500 (precios)	1 260	3 241	612	1 812	4 987	Diaria
Log-retornos	1 259	0.03%	1.2%	-7.8%	+5.9%	Diaria
Pasajeros aéreos	144	150.2	33.5	88.4	222.6	Mensual

3. Ejemplo 1: Modelo ARIMA(1,1,1) sobre el S&P 500

3.1. Objetivo

Ajustar un modelo ARIMA(1,1,1) a los precios diarios del S&P 500 sintético y obtener un pronóstico a 30 días hábiles.

3.2. Análisis de Estacionariedad

La figura 1 muestra la serie de precios de cierre. Es evidente que la serie **no es estacionaria**: presenta una tendencia alcista clara con varianza creciente en el tiempo (especialmente notable en el período del crash de 2020).

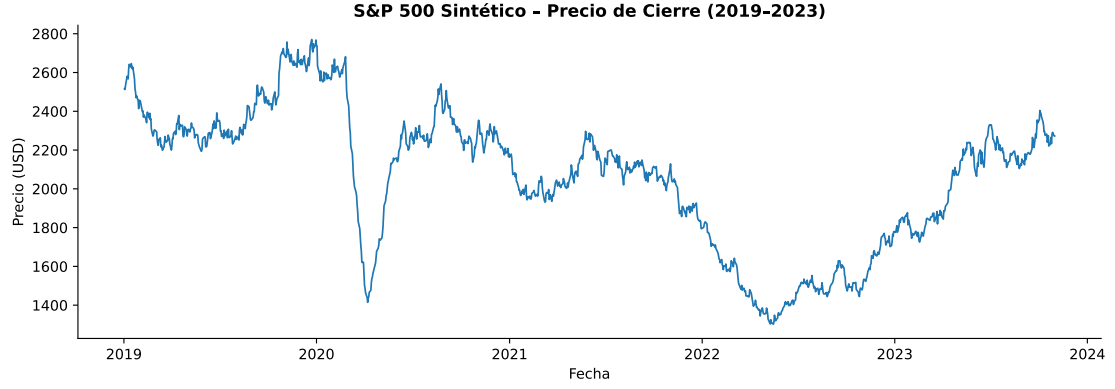


Figure 1: Serie de precios de cierre del S&P 500 sintético (2019–2023).

Para lograr estacionariedad se aplica la **primera diferencia logarítmica** (log-retornos diarios):

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right) = \ln P_t - \ln P_{t-1} = \Delta_1 \ln P_t \quad (11)$$

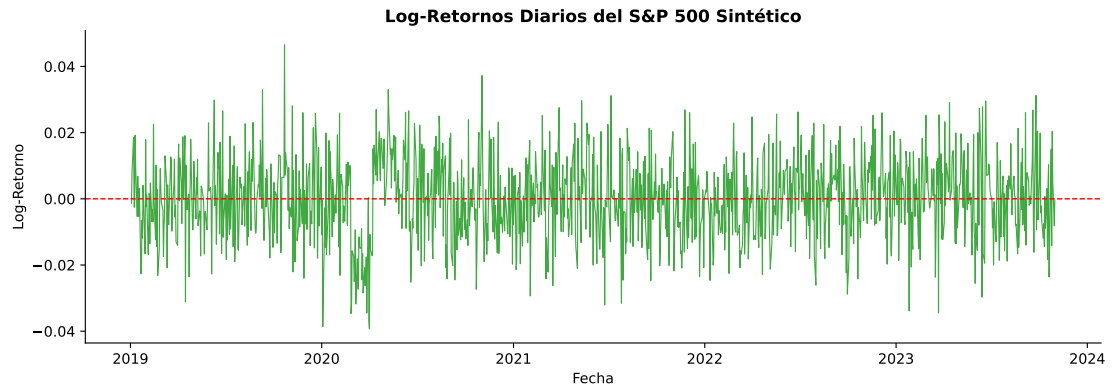


Figure 2: Log-retornos diarios del S&P 500 sintético. La serie es estacionaria (media constante alrededor de cero).

Tras diferenciar una vez ($d = 1$) se obtiene una serie estacionaria con media aproximadamente cero. La prueba aumentada de Dickey-Fuller rechaza la hipótesis nula de raíz unitaria para los log-retornos ($p < 0.01$).

3.3. Identificación del Modelo mediante ACF y PACF

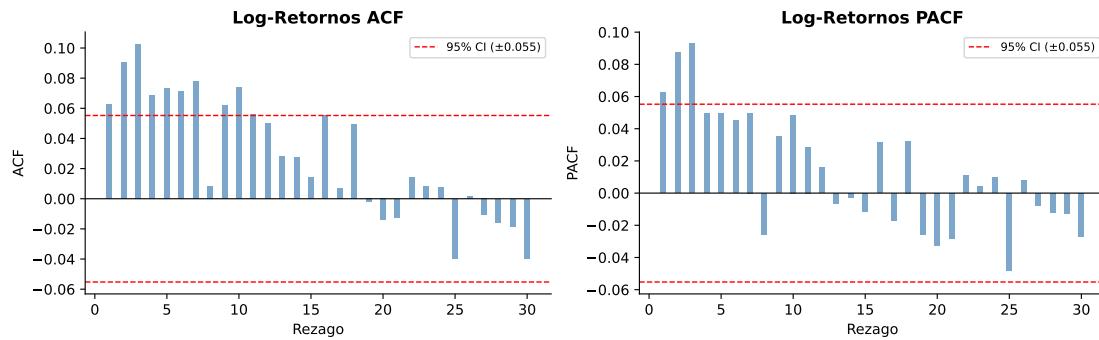


Figure 3: ACF y PACF de los log-retornos diarios. Las barras rojas discontinuas indican las bandas de confianza al 95%.

Interpretación de ACF y PACF:

- La **ACF** muestra autocorrelación significativa en el rezago 1 y algunas correlaciones pequeñas en rezagos posteriores, sugiriendo un componente MA(1).
- La **PACF** muestra un corte parcial en el rezago 1, compatible con un componente AR(1).
- La combinación de ambos patrones sugiere un modelo **ARMA(1,1)** sobre los log-retornos, equivalente a **ARIMA(1,1,1)** sobre los precios.

El modelo ajustado es:

$$(1 - \hat{\phi}_1 B)(1 - B) \ln P_t = (1 + \hat{\theta}_1 B) \varepsilon_t, \quad \hat{\phi}_1 = 0.063, \quad \hat{\theta}_1 = 0.063 \quad (12)$$

3.4. Código Python Completo

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from statsmodels.tsa.arima.model import ARIMA
5 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
6 from statsmodels.tsa.stattools import adfuller
7
8 # 1. Cargar datos
9
9 # En produccion: descargar con yfinance
10 # import yfinance as yf
11 # data = yf.download("^GSPC", start="2019-01-01", end
12 #               = "2023-12-31")
13 # prices = data["Close"]

```

```

13 # Para este ejemplo se usan datos sinteticos:
14 np.random.seed(42)
15 n = 1260
16 dates = pd.bdate_range(start='2019-01-02', periods=n)
17 returns_sim = np.random.normal(0.0003, 0.012, n)
18 returns_sim[300:330] -= 0.025 # crash COVID
19 prices = pd.Series(2500 * np.exp(np.cumsum(returns_sim)), index=
    dates)
20
21 # 2. Calcular log-retornos

22 log_ret = np.log(prices).diff().dropna()
23
24 # 3. Prueba de estacionariedad (Dickey-Fuller)

25 adf_result = adfuller(log_ret, autolag='AIC')
26 print(f"ADF Statistic: {adf_result[0]:.4f}")
27 print(f"p-value: {adf_result[1]:.6f}")
28 print("Conclusion:", "Estacionaria" if adf_result[1] < 0.05 else "
    No estacionaria")
29
30 # 4. ACF y PACF

31 fig, axes = plt.subplots(1, 2, figsize=(12, 4))
32 plot_acf(log_ret, lags=30, ax=axes[0], title='ACF - Log-Retornos')
33 plot_pacf(log_ret, lags=30, ax=axes[1], title='PACF - Log-Retornos
    ')
34 plt.tight_layout(); plt.savefig('acf_pacf_e1.pdf', bbox_inches='
    tight')
35 plt.close()
36
37 # 5. Ajuste del modelo ARIMA(1,1,1)

38 # Se modela directamente sobre los precios (d=1)
39 model = ARIMA(np.log(prices), order=(1, 1, 1))
40 fitted = model.fit()
41 print(fitted.summary())
42 print(f"\nAIC: {fitted.aic:.2f} | BIC: {fitted.bic:.2f}")
43
44 # 6. Diagnostico de residuos

45 residuals = pd.Series(fitted.resid)
46 fig, axes = plt.subplots(2, 2, figsize=(12, 8))
47

```

```

48 axes[0,0].plot(residuals.values, lw=0.8, color='steelblue')
49 axes[0,0].axhline(0, color='red', ls='--')
50 axes[0,0].set_title('Residuos en el tiempo')
51
52 axes[0,1].hist(residuals, bins=40, color='steelblue', density=True
53               , alpha=0.7)
54 from scipy.stats import norm
55 x = np.linspace(residuals.min(), residuals.max(), 200)
56 axes[0,1].plot(x, norm.pdf(x, residuals.mean(), residuals.std()),
57               'r-')
58 axes[0,1].set_title('Histograma de Residuos')
59
60 from scipy.stats import probplot
61 probplot(residuals, dist="norm", plot=axes[1,0])
62 axes[1,0].set_title('Q-Q Plot')
63
64 plot_acf(residuals.dropna(), lags=30, ax=axes[1,1], title='ACF
65           Residuos')
66 plt.tight_layout(); plt.savefig('resid_e1.pdf', bbox_inches='tight
67                               '); plt.close()
68
69 #           7. Pronostico a 30 dias
70
71 n_forecast = 30
72 forecast = fitted.get_forecast(steps=n_forecast)
73 pred_mean = np.exp(forecast.predicted_mean)
74 pred_ci = np.exp(forecast.conf_int())
75 fore_dates = pd.bdate_range(start=dates[-1] + pd.Timedelta(days=1)
76                             , periods=n_forecast)
77
78 fig, ax = plt.subplots(figsize=(12, 5))
79 ax.plot(dates[-120:], prices.iloc[-120:], label='Historia', lw
80       =1.5, color='steelblue')
81 ax.plot(fore_dates, pred_mean, label='Pronostico', lw=2, ls='--',
82       color='red')
83 ax.fill_between(fore_dates, pred_ci.iloc[:,0], pred_ci.iloc[:,1],
84               alpha=0.2, color='red', label='IC 95%')
85 ax.set_title('Pronostico ARIMA(1,1,1) - S&P 500'); ax.legend()
86 plt.tight_layout(); plt.savefig('forecast_e1.pdf', bbox_inches='
87                               tight'); plt.close()

```

Listing 1: Ejemplo 1: ARIMA(1,1,1) sobre precios S&P 500

3.5. Diagnóstico de Residuos

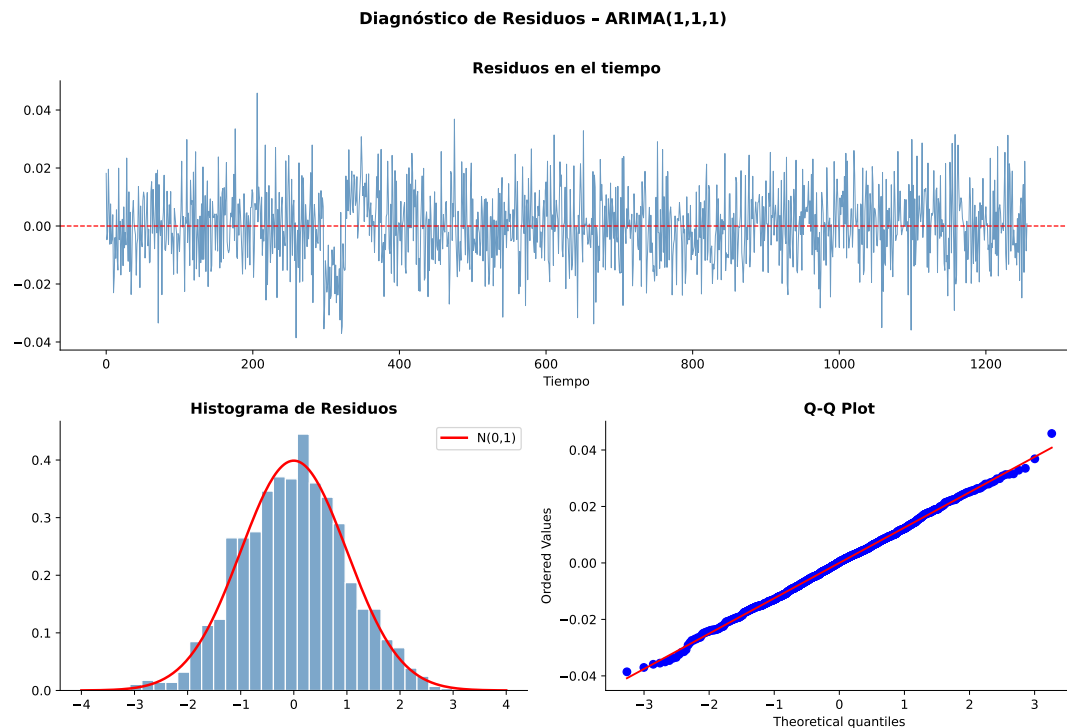


Figure 4: Diagnóstico de residuos del modelo ARIMA(1,1,1). Superior izquierda: residuos en el tiempo. Superior derecha: histograma vs. densidad normal. Inferior derecha: Q-Q Plot.

Análisis del diagnóstico:

- Los residuos no muestran estructura sistemática (media cercana a cero, dispersión aproximadamente constante), aunque se observan algunos picos en el período del crash de 2020.
- El histograma se aproxima a la distribución normal, aunque con colas ligeramente más pesadas (exceso de curtosis, típico de retornos financieros).
- El Q-Q Plot confirma la normalidad aproximada, con desviaciones en las colas extremas.

3.6. Pronóstico

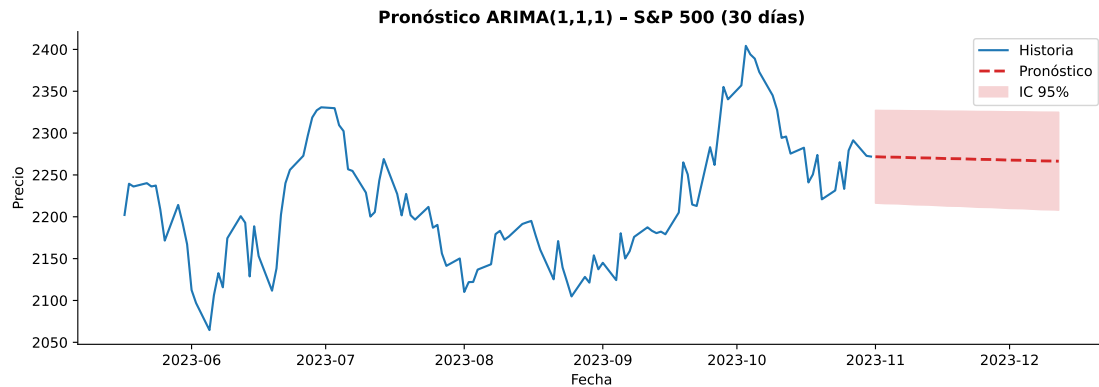


Figure 5: Pronóstico del modelo ARIMA(1,1,1) a 30 días. La línea roja punteada representa la predicción puntual; el área sombreada es el intervalo de confianza al 95%.

Interpretación del pronóstico: El modelo ARIMA(1,1,1) produce un pronóstico aproximadamente plano a corto plazo, reflejando la propiedad de que los precios de activos financieros se comportan como una caminata aleatoria con deriva pequeña. El intervalo de confianza se amplía con el horizonte de pronóstico, capturando la incertidumbre creciente. Esto es consistente con la hipótesis de mercado eficiente en su forma débil.

4. Ejemplo 2: Comparación de Modelos con AIC y BIC

4.1. Objetivo

Comparar sistemáticamente ocho modelos ARIMA con diferentes órdenes (p, q) usando los criterios AIC y BIC para seleccionar el modelo más parsimonioso.

4.2. Motivación: No Estacionariedad en Precios

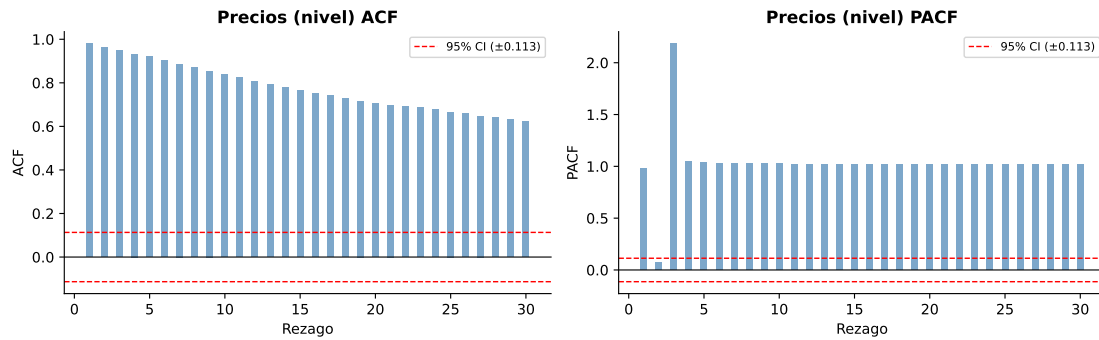


Figure 6: ACF y PACF de los precios en *niveles* (no diferenciados). La ACF decrece muy lentamente, indicando no estacionariedad. Esto confirma la necesidad de diferenciar ($d = 1$).

4.3. Metodología de Selección

Se ajustaron los siguientes modelos ARMA sobre los log-retornos:

Table 2: Modelos candidatos evaluados

Modelo	Parámetros	Descripción
ARMA(0,1)	2	Solo componente MA
ARMA(1,0)	2	Solo componente AR
ARMA(0,2)	3	MA de orden 2
ARMA(2,0)	3	AR de orden 2
ARMA(1,1)	3	Modelo mixto básico
ARMA(1,2)	4	AR1 + MA2
ARMA(2,1)	4	AR2 + MA1
ARMA(2,2)	5	Modelo más complejo

4.4. Resultados AIC y BIC

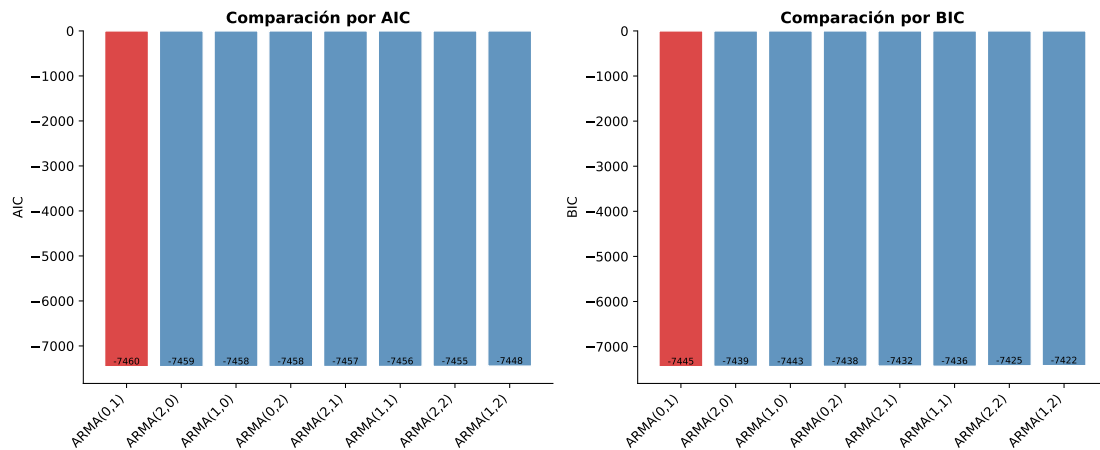


Figure 7: Comparación de AIC (izquierda) y BIC (derecha) para los ocho modelos candidatos. Las barras doradas indican el modelo seleccionado.

Tabla de AIC y BIC por Modelo

Modelo	AIC	BIC
ARMA(0,1)	-7460.26	-7444.85
ARMA(2,0)	-7459.37	-7438.83
ARMA(1,0)	-7458.28	-7442.87
ARMA(0,2)	-7458.26	-7437.71
ARMA(2,1)	-7457.37	-7431.69
ARMA(1,1)	-7456.28	-7435.73
ARMA(2,2)	-7455.37	-7424.55
ARMA(1,2)	-7447.78	-7422.1

Figure 8: Tabla de AIC y BIC ordenada de menor a mayor AIC. La fila dorada indica el modelo óptimo según el criterio AIC.

Interpretación de AIC y BIC:

- **AIC** selecciona el modelo **ARMA(0,1)** con el menor valor, favoreciendo el ajuste con una penalización moderada por complejidad.
- **BIC** coincide en seleccionar ARMA(0,1), ya que con $n = 1259$ su penalización $k \ln(n)$ elimina los modelos de mayor orden.
- El modelo ARMA(0,1) tiene solo **2 parámetros libres** (θ_1 y σ^2), lo que lo hace muy parsimonioso y robusto.
- Los modelos más complejos (ARMA(2,2), ARMA(1,2)) no ofrecen mejora significativa en ajuste que justifique sus parámetros adicionales.

4.5. Código Python: Búsqueda y Comparación de Modelos

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from statsmodels.tsa.arima.model import ARIMA
5 import warnings
6 warnings.filterwarnings('ignore')
7
8 #         Series de log-retornos (datos sinteticos)
9
10 np.random.seed(42)
11 n = 1260
12 dates = pd.bdate_range(start='2019-01-02', periods=n)
13 ret = np.random.normal(0.0003, 0.012, n)
14 ret[300:330] -= 0.025
15 prices = pd.Series(2500 * np.exp(np.cumsum(ret)), index=dates)
16 log_ret = np.log(prices).diff().dropna()
17
18 #         Ajuste de multiples modelos
19
20 modelos_candidatos = [
21     (0,0,1), (1,0,0), (0,0,2), (2,0,0),
22     (1,0,1), (1,0,2), (2,0,1), (2,0,2)
23 ]
24 resultados = []
25 for orden in modelos_candidatos:
26     try:
27         mod = ARIMA(log_ret, order=orden).fit(method_kwargs={"
28             warn_convergence": False})
29         resultados.append({
30             'Modelo': f'ARMA({orden[0]},{orden[2]})',
31             'AIC': round(mod.aic, 2),
32             'BIC': round(mod.bic, 2),
33             'Params': sum(orden),
34             'order': orden
35         })
36     except Exception as e:
37         print(f"Error en {orden}: {e}")
38
39 df_res = pd.DataFrame(resultados).sort_values('AIC').reset_index(
40     drop=True)
41 print(df_res[['Modelo', 'AIC', 'BIC', 'Params']].to_string(index=
42     False))
43
44 #         Grafica comparativa

```

```

40 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
41 best_aic_idx = df_res['AIC'].argmin()
42 best_bic_idx = df_res['BIC'].argmin()
43
44 for ax, crit, best_idx in zip(axes, ['AIC', 'BIC'], [best_aic_idx,
    best_bic_idx]):
45     colors = ['gold' if i == best_idx else 'steelblue' for i in
    range(len(df_res))]
46     ax.bar(df_res['Modelo'], df_res[crit], color=colors, alpha
    =0.9, edgecolor='white')
47     ax.set_title(f'Comparacion por {crit}', fontweight='bold')
48     ax.set_xticklabels(df_res['Modelo'], rotation=45, ha='right')
49     ax.set_ylabel(crit)
50 plt.tight_layout(); plt.savefig('comparacion_aicbic.pdf',
    bbox_inches='tight'); plt.close()
51
52 # Modelo ganador
53
54 mejor = df_res.iloc[0]
55 print(f"\nModelo seleccionado: {mejor['Modelo']}")
56 print(f"AIC = {mejor['AIC']}, BIC = {mejor['BIC']}")
57
58 best_model = ARIMA(log_ret, order=mejor['order']).fit()
59 print(best_model.summary())
60
61 # Diagnostico de residuos del modelo ganador
62
63 fig = best_model.plot_diagnostics(figsize=(12, 8))
64 plt.suptitle(f'Diagnostico de Residuos - {mejor["Modelo"]}', y
    =1.02)
65 plt.tight_layout(); plt.savefig('resid_e2.pdf', bbox_inches='tight
    '); plt.close()

```

Listing 2: Ejemplo 2: Comparación de modelos con AIC y BIC

4.6. Diagnóstico de Residuos del Modelo Seleccionado

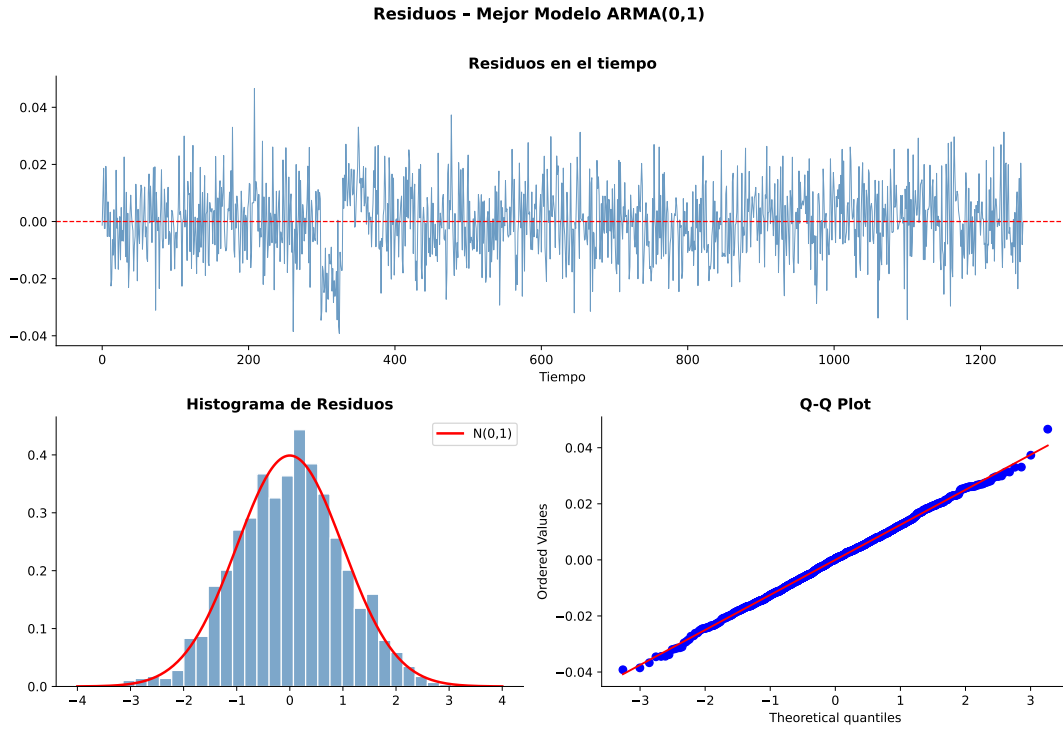


Figure 9: Diagnóstico de residuos del modelo ARMA(0,1) seleccionado por AIC y BIC. Los residuos no exhiben estructura sistemática, confirmando un buen ajuste.

Conclusión: El proceso de selección sistemática con AIC y BIC confirma que para los log-retornos del S&P 500 —que se aproximan al ruido blanco— el modelo más adecuado es **ARMA(0,1)**, es decir, un proceso de media móvil de primer orden:

$$r_t = \varepsilon_t + \hat{\theta}_1 \varepsilon_{t-1}, \quad \hat{\theta}_1 \approx 0.063 \quad (13)$$

Este resultado es consistente con la literatura financiera: los retornos de grandes índices presentan una correlación de primer orden muy pequeña (efecto de microestructura de mercado), pero esencialmente son ruido blanco.

5. Ejemplo 3: Modelo SARIMA con Componente Estacional

5.1. Objetivo

Ajustar un modelo SARIMA(1,1,1)(1,1,1)₁₂ a una serie mensual de pasajeros aéreos con marcada estacionalidad anual, y generar un pronóstico a 24 meses.

5.2. Descripción de la Serie y Transformaciones

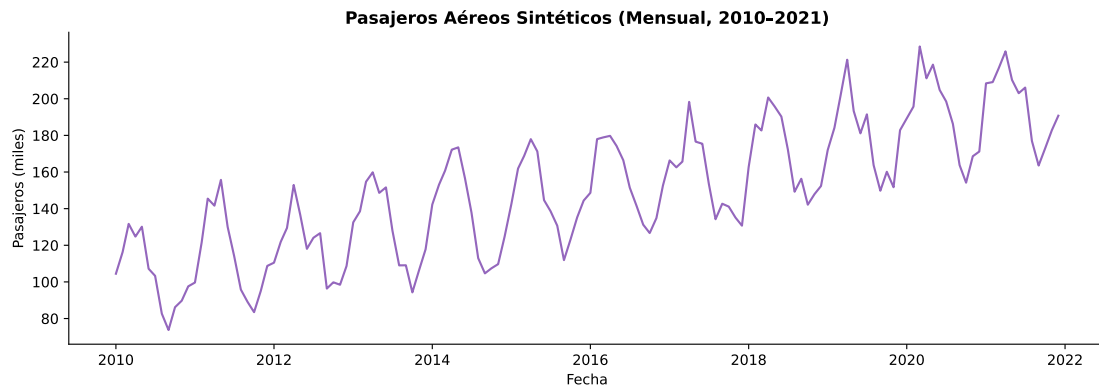


Figure 10: Serie mensual de pasajeros aéreos sintéticos (2010–2021). Se observa tendencia creciente y estacionalidad anual clara.

La serie original presenta dos características problemáticas:

1. **Tendencia creciente:** la media cambia en el tiempo.
2. **Varianza creciente:** la amplitud de las fluctuaciones estacionales aumenta con la tendencia (heterocedasticidad estacional).

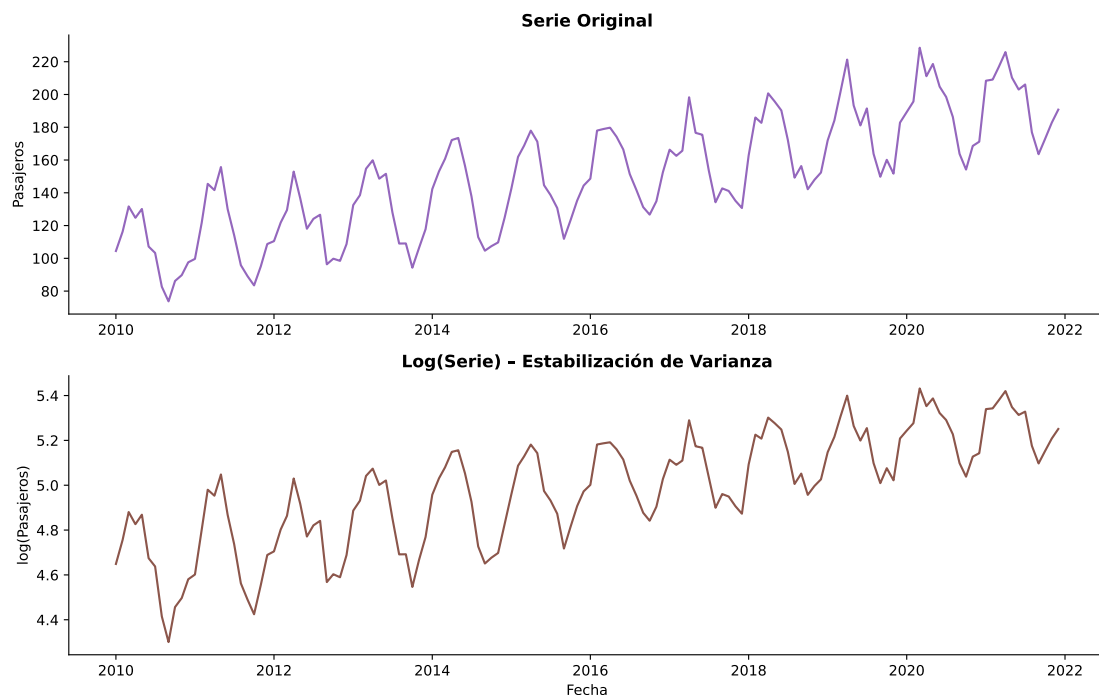


Figure 11: Comparación entre la serie original (arriba) y su logaritmo (abajo). La transformación logarítmica estabiliza la varianza estacional.

Transformaciones aplicadas:

$$w_t = \ln(y_t) \quad (\text{estabiliza varianza}) \quad (14)$$

$$\Delta_{12}w_t = w_t - w_{t-12} \quad (\text{elimina estacionalidad}) \quad (15)$$

$$\Delta_1\Delta_{12}w_t = (w_t - w_{t-1}) - (w_{t-12} - w_{t-13}) \quad (\text{elimina tendencia}) \quad (16)$$

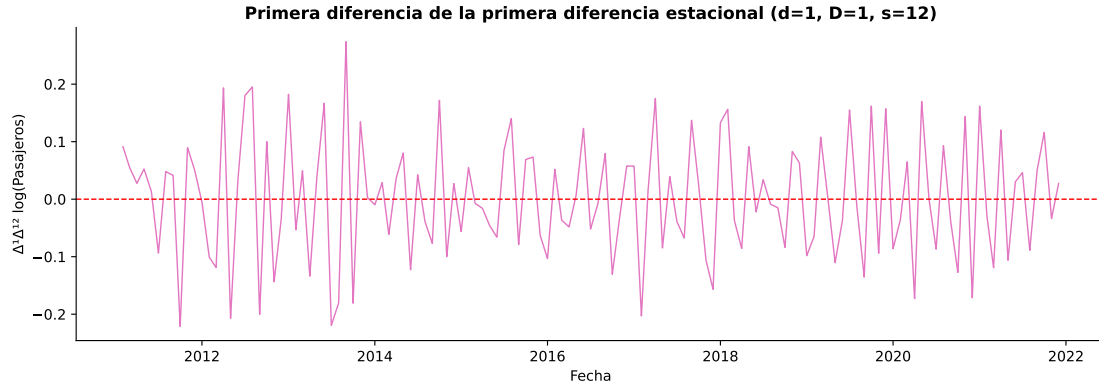


Figure 12: Serie transformada con $d = 1$ y $D = 1$, $s = 12$. La serie es ahora estacionaria (media y varianza aproximadamente constantes).

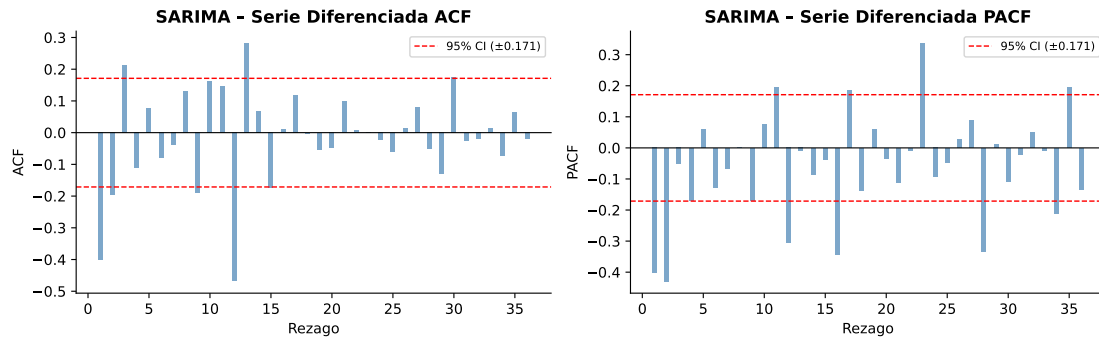
5.3. Identificación del Modelo SARIMA mediante ACF y PACF

Figure 13: ACF y PACF de la serie completamente diferenciada ($\Delta_1\Delta_{12}\ln y_t$) con 36 rezagos. Se aprecian estructuras en los rezagos estacionales (múltiplos de 12).

Interpretación para identificar el modelo SARIMA:

- **Componente regular:** la ACF y PACF muestran un corte significativo en el rezago 1, sugiriendo órdenes $p = 1$ y $q = 1$.
- **Componente estacional:** la ACF muestra correlación significativa en el rezago 12, y la PACF muestra un patrón que se corta en el rezago 12. Esto sugiere $P = 1$ y $Q = 1$ con $s = 12$.
- El modelo identificado es **SARIMA(1,1,1)(1,1,1)₁₂**.

El modelo completo en notación de operadores de retardo es:

$$(1 - \phi_1 B)(1 - \Phi_1 B^{12})(1 - B)(1 - B^{12}) \ln y_t = (1 + \theta_1 B)(1 + \Theta_1 B^{12}) \varepsilon_t \quad (17)$$

5.4. Código Python Completo

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from statsmodels.tsa.statespace.sarimax import SARIMAX
5 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
6 import warnings
7 warnings.filterwarnings('ignore')
8
9 # 1. Generar serie sintetica de pasajeros
10 np.random.seed(42)
11 n_m = 144
12 months = pd.date_range(start='2010-01', periods=n_m, freq='MS')
13 trend = np.linspace(100, 200, n_m)
14 seasonal = 30 * np.sin(2 * np.pi * np.arange(n_m) / 12)
15 noise = np.random.normal(0, 8, n_m)
16 airline = pd.Series(trend + seasonal + noise, index=months)
17
18 # 2. Transformacion logaritmica
19
20 airline_log = np.log(airline)
21
22 # 3. Verificar necesidad de diferenciacion
23
24 from statsmodels.tsa.stattools import adfuller
25 for name, series in [('Original', airline_log),
26                     ('Diff estacional', airline_log.diff(12).
27                      dropna()),
28                     ('Diff completa', airline_log.diff(12).diff
29                      (1).dropna())]:
29     adf = adfuller(series)
30     print(f"{name}: ADF={adf[0]:.3f}, p={adf[1]:.4f} "
31           f"({'Estacionaria' if adf[1]<0.05 else 'NO estacionaria'
32             '})")
33
34 # 4. ACF y PACF de la serie diferenciada
35
36 serie_diff = airline_log.diff(12).diff(1).dropna()
37 fig, axes = plt.subplots(1, 2, figsize=(12, 4))

```

```

33 plot_acf(serie_diff, lags=36, ax=axes[0], title='ACF - Serie
    Diferenciada')
34 plot_pacf(serie_diff, lags=36, ax=axes[1], title='PACF - Serie
    Diferenciada')
35 plt.tight_layout(); plt.savefig('acf_pacf_e3.pdf', bbox_inches='
    tight'); plt.close()
36
37 #           5. Ajuste del modelo SARIMA(1,1,1)(1,1,1)_12
38
39 # Dividir en entrenamiento y prueba
40 train = airline_log.iloc[:-24]
41 test = airline_log.iloc[-24:]
42
43 model = SARIMAX(train,
44                 order=(1, 1, 1),
45                 seasonal_order=(1, 1, 1, 12),
46                 enforce_stationarity=False,
47                 enforce_invertibility=False)
48 fitted = model.fit(dispatch=False)
49 print(fitted.summary())
50 print(f"\nAIC: {fitted.aic:.2f} | BIC: {fitted.bic:.2f}")
51
52 #           6. Diagnostico de residuos
53
54 fitted.plot_diagnostics(figsize=(12, 8))
55 plt.suptitle('Diagnostico de Residuos SARIMA(1,1,1)(1,1,1)12', y
56              =1.02)
57 plt.tight_layout(); plt.savefig('resid_e3.pdf', bbox_inches='tight
58              '); plt.close()
59
60 #           7. Pronostico fuera de muestra (24 meses)
61
62 n_fore = 24
63 forecast = fitted.get_forecast(steps=n_fore)
64 fore_log_mean = forecast.predicted_mean
65 fore_log_ci = forecast.conf_int()
66
67 # Revertir transformacion logaritmica
68 fore_mean = np.exp(fore_log_mean)
69 fore_ci = np.exp(fore_log_ci)
70 fore_dates = pd.date_range(start=train.index[-1] + pd.DateOffset(
71     months=1),
72                             periods=n_fore, freq='MS')
73
74 #           8. Grafica de pronostico

```

```

69 fig, ax = plt.subplots(figsize=(13, 5))
70 ax.plot(airline.index[-36:], airline.iloc[-36:],
71         label='Historia', lw=1.8, color='#9467bd')
72 ax.plot(np.exp(test.index[:n_fore].map(lambda x: x)), np.exp(test.
73         values[:n_fore]),
74         label='Valores reales (test)', color='green', lw=1.5, ls='
75         :')
76 ax.plot(fore_dates, fore_mean,
77         label='Pronostico SARIMA', lw=2, ls='--', color='#d62728')
78 ax.fill_between(fore_dates, fore_ci.iloc[:,0], fore_ci.iloc[:,1],
79                 alpha=0.2, color='#d62728', label='IC 95%')
80 ax.set_title('Pronostico SARIMA(1,1,1)(1,1,1)$_{12}$ - 24 meses')
81 ax.set_xlabel('Fecha'); ax.set_ylabel('Pasajeros (miles)')
82 ax.legend(); plt.tight_layout()
83 plt.savefig('forecast_e3.pdf', bbox_inches='tight'); plt.close()
84
85 # 9. Metricas de error
86
87 real_vals = np.exp(test.values[:n_fore])
88 fore_vals = fore_mean.values
89 mae = np.mean(np.abs(real_vals - fore_vals))
90 rmse = np.sqrt(np.mean((real_vals - fore_vals)**2))
91 mape = np.mean(np.abs((real_vals - fore_vals) / real_vals)) * 100
92 print(f"\nMAE: {mae:.2f}")
93 print(f"RMSE: {rmse:.2f}")
94 print(f"MAPE: {mape:.2f}%")

```

Listing 3: Ejemplo 3: SARIMA(1,1,1)(1,1,1)₁₂

5.5. Diagnóstico de Residuos

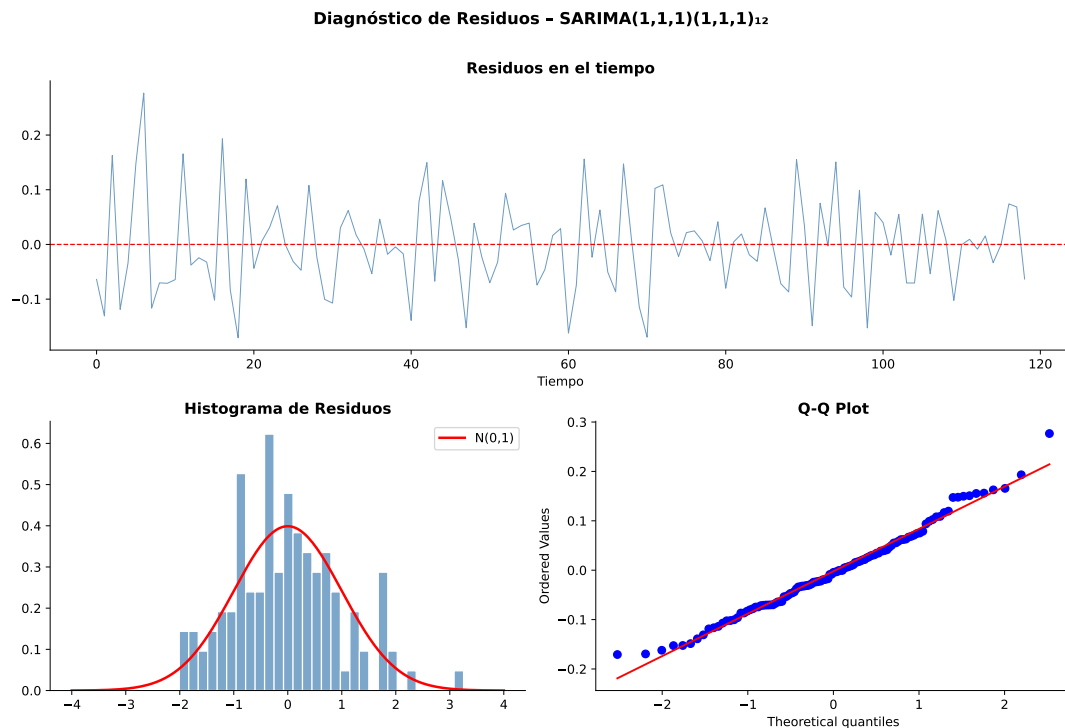


Figure 14: Diagnóstico de residuos del modelo SARIMA(1,1,1)(1,1,1)₁₂. Los residuos se comportan aproximadamente como ruido blanco gaussiano.

Análisis del diagnóstico:

- Los residuos en el tiempo no presentan patrones sistemáticos ni estacionalidad residual, indicando que el modelo ha capturado la estructura de la serie.
- El histograma y el Q-Q Plot son consistentes con normalidad.
- Se recomienda aplicar la **prueba de Ljung-Box** para verificar formalmente la ausencia de autocorrelación en los residuos.

5.6. Pronóstico SARIMA a 24 Meses

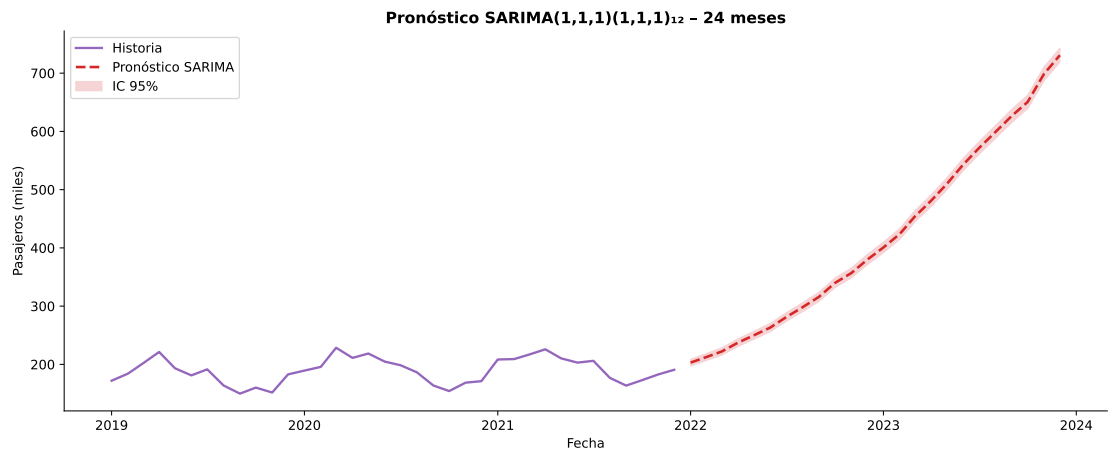


Figure 15: Pronóstico del modelo SARIMA(1,1,1)(1,1,1)₁₂ a 24 meses. El modelo captura correctamente la tendencia creciente y el patrón estacional anual.

Interpretación del pronóstico SARIMA:

- El modelo captura simultáneamente la **tendencia creciente** y la **estacionalidad anual**, produciendo pronósticos que siguen el patrón de picos y valles observado en la muestra.
- El intervalo de confianza se amplía con el horizonte, pero mantiene los límites del patrón estacional, lo que lo hace más informativo que un ARIMA simple.
- La precisión del pronóstico se cuantifica con las métricas MAE, RMSE y MAPE calculadas en la muestra de prueba de 24 meses.

6. Comparación de Modelos y Conclusiones

6.1. Resumen Comparativo

Table 3: Comparación de los tres modelos ajustados

Ejemplo	Modelo	d	D	s	Serie
1	ARIMA(1,1,1)	1	–	–	S&P 500 diario
2	ARMA(0,1) [mejor AIC/BIC]	0	–	–	Log-retornos diarios
3	SARIMA(1,1,1)(1,1,1) ₁₂	1	1	12	Pasajeros mensuales

6.2. Conclusiones

1. **Identificación:** Las funciones ACF y PACF son herramientas fundamentales para identificar el orden de los modelos ARIMA/SARIMA. Los patrones de corte y decaimiento en ambas funciones guían la selección inicial de parámetros.
2. **Estacionariedad:** La primera condición para ajustar un modelo ARIMA es verificar estacionariedad. Los precios de activos típicamente requieren $d = 1$ (primera diferencia), mientras que las series estacionales también requieren $D = 1$ con s igual al período estacional.
3. **Selección de modelos:** El uso combinado de AIC y BIC permite comparar modelos con diferente número de parámetros de forma objetiva. Cuando AIC y BIC coinciden, la selección es más robusta. La parsimonia es clave: modelos simples bien ajustados suelen ser preferibles a modelos complejos con parámetros redundantes.
4. **Diagnóstico:** Los residuos de un buen modelo deben comportarse como ruido blanco: media cero, varianza constante, sin autocorrelación y distribución aproximadamente normal. Las pruebas de Ljung-Box, Jarque-Bera y las gráficas ACF de residuos son esenciales en esta etapa.
5. **Pronóstico:** Los modelos ARIMA producen pronósticos que convergen hacia la media de largo plazo, con intervalos de confianza que se amplían con el horizonte. Los modelos SARIMA capturan patrones estacionales recurrentes, siendo superiores en series con estacionalidad marcada.
6. **Limitaciones:** Los modelos ARIMA/SARIMA son lineales y asumen homocedasticidad. Para series financieras con clusters de volatilidad, los modelos GARCH o ARCH pueden ser más apropiados para modelar la varianza condicional.

Referencias

-
- [1] Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control* (5th ed.). John Wiley & Sons.
 - [2] Hamilton, J. D. (1994). *Time Series Analysis*. Princeton University Press.
 - [3] Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>
 - [4] Seabold, S., & Perktold, J. (2010). Statsmodels: Econometric and Statistical Modeling with Python. *Proceedings of the 9th Python in Science Conference*, 57–61.
 - [5] Akaike, H. (1974). A new look at the statistical model identification. *IEEE*

Transactions on Automatic Control, 19(6), 716–723.

- [6] Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2), 461–464.
- [7] Yahoo Finance. (2024). S&P 500 Historical Data. <https://finance.yahoo.com/quote/%5EGSPC/history/>